基于 C 的 MDK 工程建立及跟踪和调试过程的记录

1. 观察以下变量存放格式并记录。

```
int main (void)
{
unsigned int ui_tmp;
unsigned int ui_a, ui_b, ui_c;
static int i_tmp;
//signed int (32bits)
static short s16_tmp;
//signed short (16bits)
static float f_tmp;
//floating point (32bits)
static int s[8];
int k;
//记录浮点数 IEEE754 规范表示方法
f_tmp = -0.5;
f_tmp = f_tmp + 1;
//临时变量，观察 ui_tmp, ui_a, ui_b, ui_c 被保存在哪里
//观察执行前后 PC 寄存器
ui_a = 1;
ui_b = 2;
ui_c = 0xFF;
//观察执行后 PSR 寄存器的标志位 Negtive
ui_c = ui_a - ui_b;
ui_tmp = ui_c;//观察数的表示方法（补码）
i_tmp = -1;
i_tmp = i_tmp - 1;
s16_tmp = -1;
s16_tmp = -2;
s16_tmp = s16_tmp - 32766;
//单步跟踪观察循环体执行过程
for(k=8; k>0; --k)
s[k-1] = 0x80000000 + k;
}
```

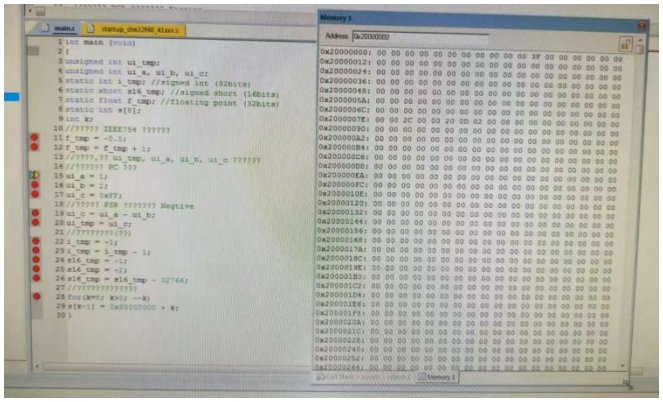
分析如下：

该代码段展示了多种变量类型的声明和赋值操作，并涉及对这些变量的观察、修改以及与浮点数和整数运算相关的操作。分析一下各个部分：

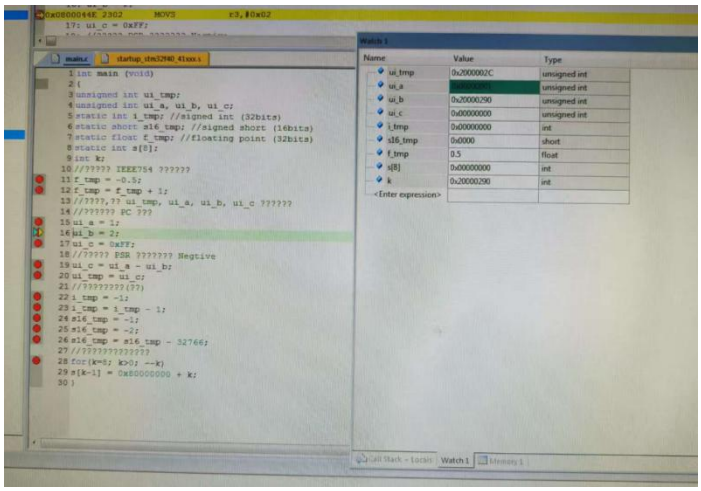
变量声明及存放格式：

unsigned int ui_tmp, ui_a, ui_b, ui_c;

类型：unsigned int，无符号整数，占用 4 字节（32 位）。
这些变量存储的是无符号整数，最大值为 0xFFFFFFFF，最小值为 0。
static int i_tmp;
类型：int，带符号整数，占用 4 字节（32 位）。
由于使用了 static 关键字，i_tmp 的生命周期是程序运行期间，且该变量只会初始化一次。它存储的是带符号整数。
static short s16_tmp;
类型：short，带符号短整数，占用 2 字节（16 位）。
同样使用了 static 关键字，生命周期与 i_tmp 相同，存储的是 16 位的带符号整数。
static float f_tmp;
类型：float，浮点数，占用 4 字节（32 位）。
存储浮点数，根据 IEEE 754 标准，f_tmp 会存储单精度浮点数。
static int s[8];
类型：int[8]，一个整型数组，包含 8 个 int 类型的元素，每个元素占用 4 字节，共占用 32 字节。
int k;
类型：int，普通整数，占用 4 字节（32 位）。
通过断点步进执行可得到如下记录结果：
在其分析如下；
//记录浮点数 IEEE754 规范表示方法
f_tmp = -0.5; f_tmp = f_tmp + 1;
符号位 1，指数位-1+偏移量 127，尾数 23 位全为 0，16 进制下即为 0xBF000000
//临时变量，观察 ui_tmp, ui_a, ui_b, ui_c 被保存在哪里
保存在 R2-R5 寄存器中
//观察执行前后 PC 寄存器 PC 寄存器显示的是正在执行的变量的地址
ui_a = 1;
ui_b = 2;
ui_c = 0xFF;
//观察执行后 PSR 寄存器的标志位 Negtive N 位由 0 变为 1
ui_c = ui_a - ui_b;
ui_tmp = ui_c;
//观察数的表示方法（补码）
i_tmp = -1; 0xFFFFFFFF
i_tmp = i_tmp - 1; 0xFFFFFFFE
s16_tmp = -1; 0xFFFF
s16_tmp = -2; 0xFFFE
s16_tmp = s16_tmp - 32766; 0x8000
//单步跟踪观察循环体执行过程
for(k=8; k>0; --k) s[k-1] = 0x80000000 + k; }

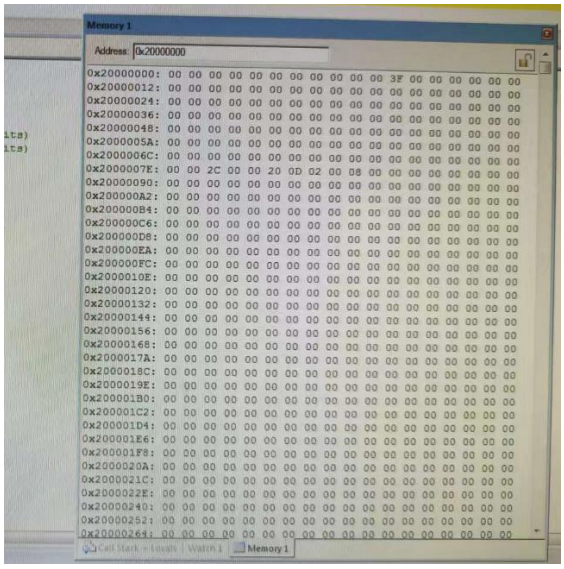


```
1 int main (void)
2 {
3 unsigned int ui_tmp;
4 unsigned int ui_a, ui_b, ui_c;
5 static int i_tmp; //signed int (32bits)
6 static short s16_tmp; //signed short (16bits)
7 static float f_tmp; //floating point (32bits)
8 static int s[8];
9 int k;
10 //????? IEEE754 ??????
11 f_tmp = -0.5;
12 f_tmp = f_tmp + 1;
13 //????,?? ui_tmp, ui_a, ui_b, ui_c ??????
14 //??????? PC ???
15 ui_a = 1;
16 ui_b = 2;
17 ui_c = 0xFF;
18 //????? PSR ??????? Negtive
19 ui_c = ui_a - ui_b;
20 ui_tmp = ui_c;
21 //????????(??)
22 i_tmp = -1;
23 i_tmp = i_tmp - 1;
24 s16_tmp = -1;
25 s16_tmp = -2;
26 s16_tmp = s16_tmp - 32766;
27 //????????????
28 for(k=8; k>0; --k)
29 s[k-1] = 0x80000000 + k;
30 }
```



```
0x0800064E 2302      MOVS      r3,#0x02
17: ui_c = 0xFF;
1 int main (void)
2 {
3 unsigned int ui_tmp;
4 unsigned int ui_a, ui_b, ui_c;
5 static int i_tmp; //signed int (32bits)
6 static short s16_tmp; //signed short (16bits)
7 static float f_tmp; //floating point (32bits)
8 static int s[8];
9 int k;
10 //????? IEEE754 ??????
11 f_tmp = -0.5;
12 f_tmp = f_tmp + 1;
13 //????,?? ui_tmp, ui_a, ui_b, ui_c ??????
14 //??????? PC ???
15 ui_a = 1;
16 ui_b = 2;
17 ui_c = 0xFF;
18 //????? PSR ??????? Negtive
19 ui_c = ui_a - ui_b;
20 ui_tmp = ui_c;
21 //????????(??)
22 i_tmp = -1;
23 i_tmp = i_tmp - 1;
24 s16_tmp = -1;
25 s16_tmp = -2;
26 s16_tmp = s16_tmp - 32766;
27 //????????????
28 for(k=8; k>0; --k)
29 s[k-1] = 0x80000000 + k;
30 }
```

Name	Value	Type
ui_tmp	0x2000002C	unsigned int
ui_a	0x00000001	unsigned int
ui_b	0x20000290	unsigned int
ui_c	0x00000000	unsigned int
i_tmp	0x00000000	int
s16_tmp	0x0000	short
f_tmp	0.5	float
s[8]	0x00000000	int
k	0x20000290	int
<Enter expression>		

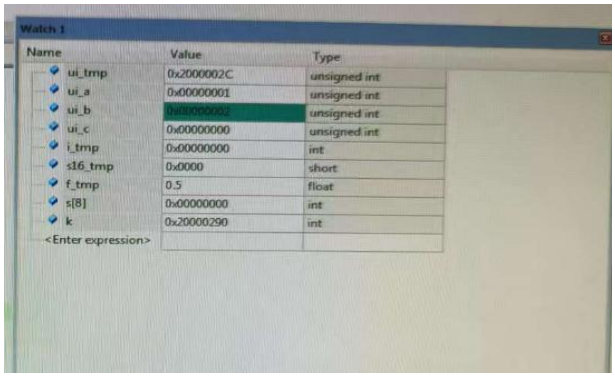


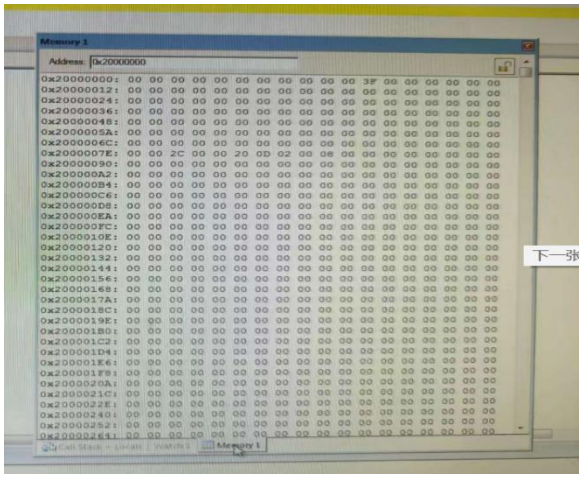
Memory 1

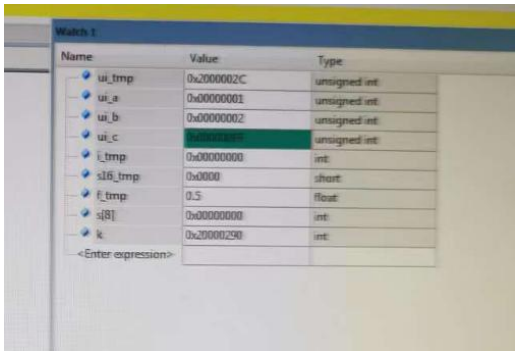
Address: 0x20000000

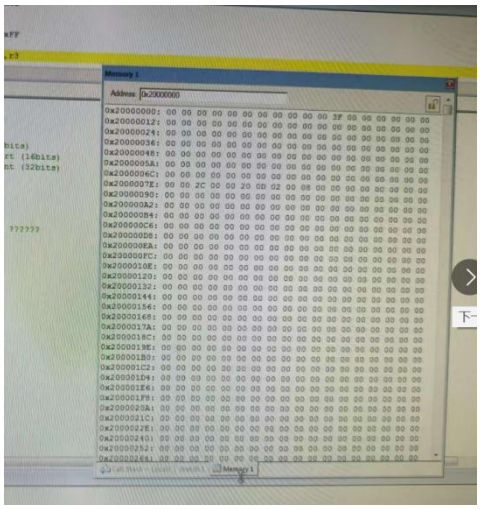
```
0x20000000: 00 00 00 00 00 00 00 00 00 00 00 00 3F 00 00 00 00 00 00 00
0x20000012: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000024: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000036: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000048: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000005A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000006C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000007E: 00 00 2C 00 00 20 0D 02 00 08 00 00 00 00 00 00 00 00 00 00
0x20000090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000A2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000B4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000C6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000D8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000EA: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000FC: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000010E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000132: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000144: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000156: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000168: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000017A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000018C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000019E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001C2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001D4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001E6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001F8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000020A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000021C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000022E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000240: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000252: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000264: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

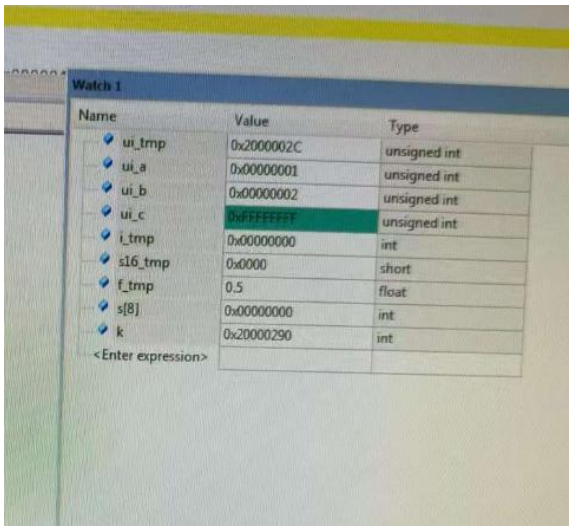
Call Stack + Locals | Watch 1 | Memory 1

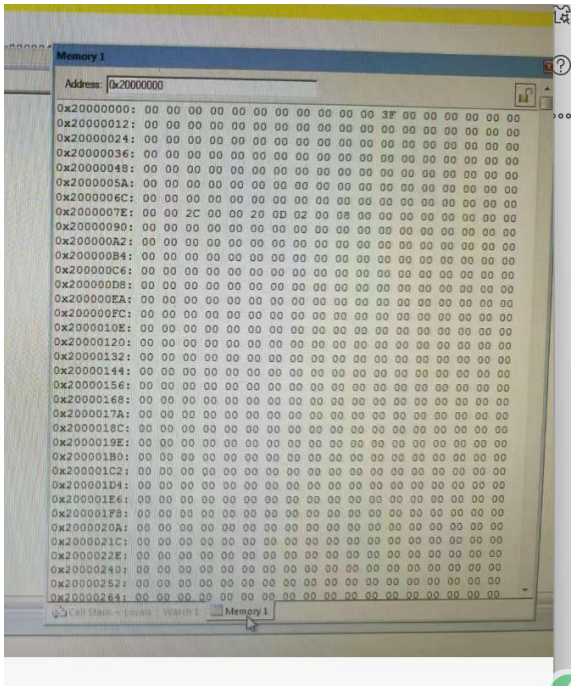

Watch 1
Name
Value
Type
ui_tmp
0x2000002C
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0x00000000
unsigned int
i_tmp
0x00000000
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>

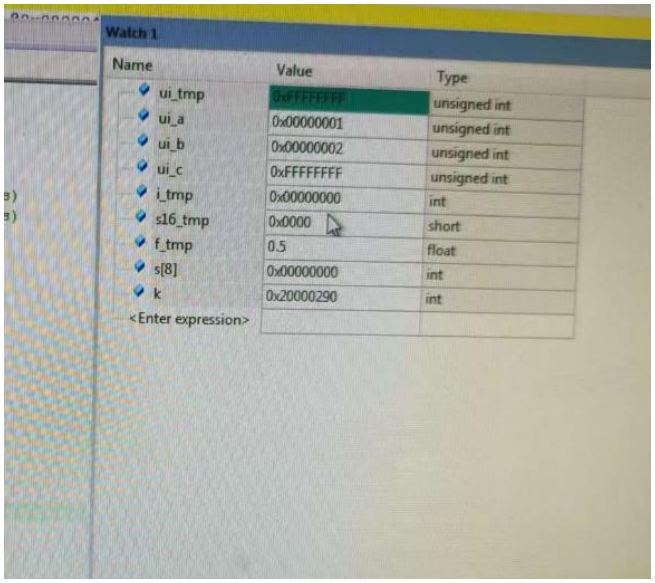

Memory 1
Address: 0x20000000

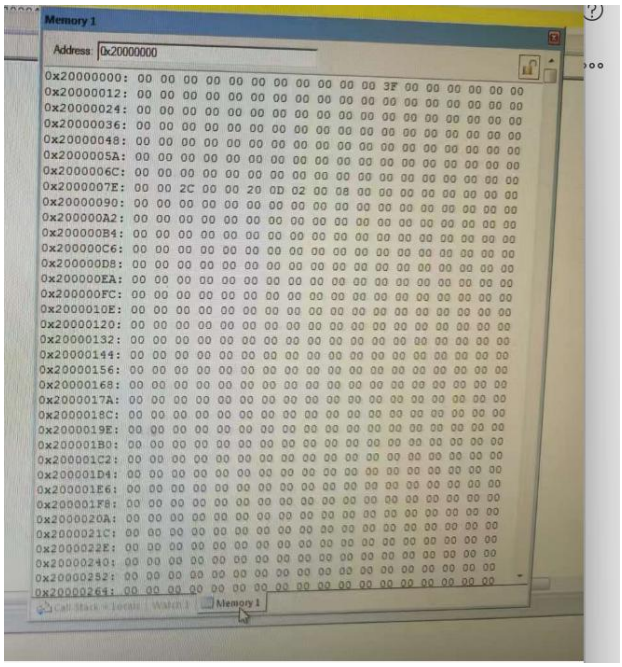

Watch 1
Name
Value
Type
ui_tmp
0x2000002C
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
unsigned int
i_tmp
0x00000000
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>

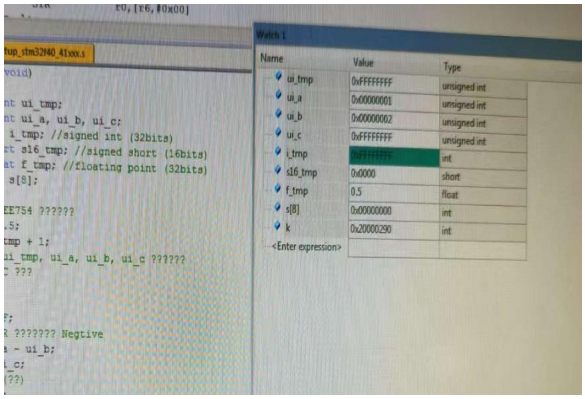

Memory 1
Address: 0x20000000

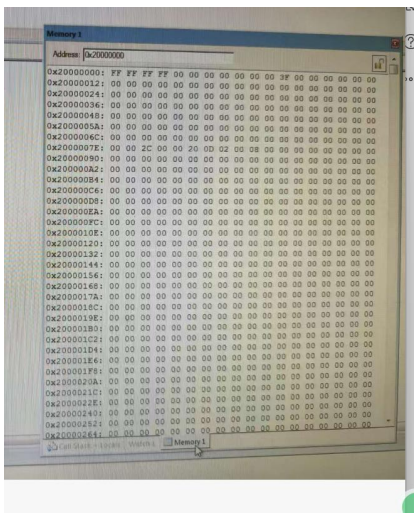

Watch 1
Name
Value
Type
ui_tmp
0x2000002C
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0xFFFFFFFF
unsigned int
i_tmp
0x00000000
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>

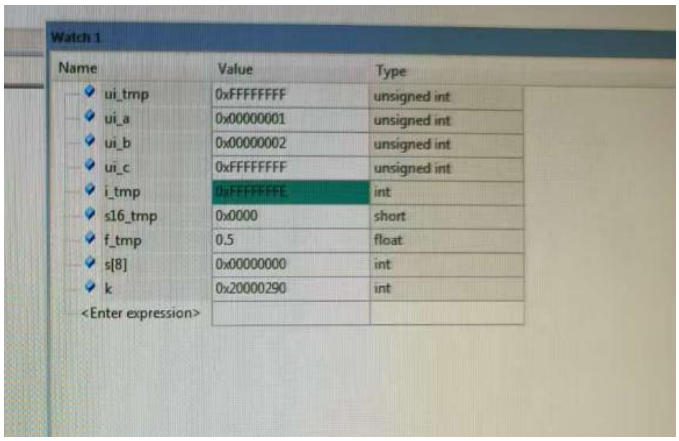

Memory 1
Address: 0x20000000
Memory 1

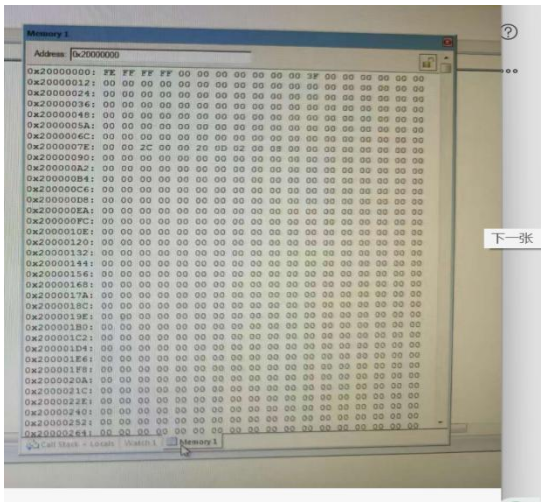

Watch 1
Name
Value
Type
ui_tmp
0xFFFFFFFF
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0xFFFFFFFF
unsigned int
i_tmp
0x00000000
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>

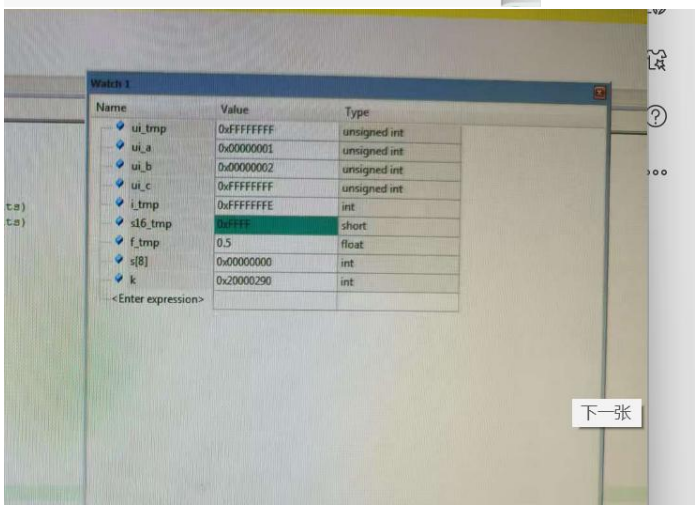

Memory 1
Address: 0x20000000
0x20000000: 00 00 00 00 00 00 00 00 00 00 00 00 3F 00 00 00 00 00 00 00
0x20000012: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000024: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000036: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000048: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000005A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000006C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000007E: 00 00 2C 00 00 20 0D 02 00 08 00 00 00 00 00 00 00 00 00 00
0x20000090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000A2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000B4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000C6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000D8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000EA: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000FC: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000010E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000132: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000144: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000156: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000168: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000017A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000018C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000019E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001C2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001D4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001E6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001F8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000020A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000021C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000022E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000240: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000252: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000264: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Memory 1

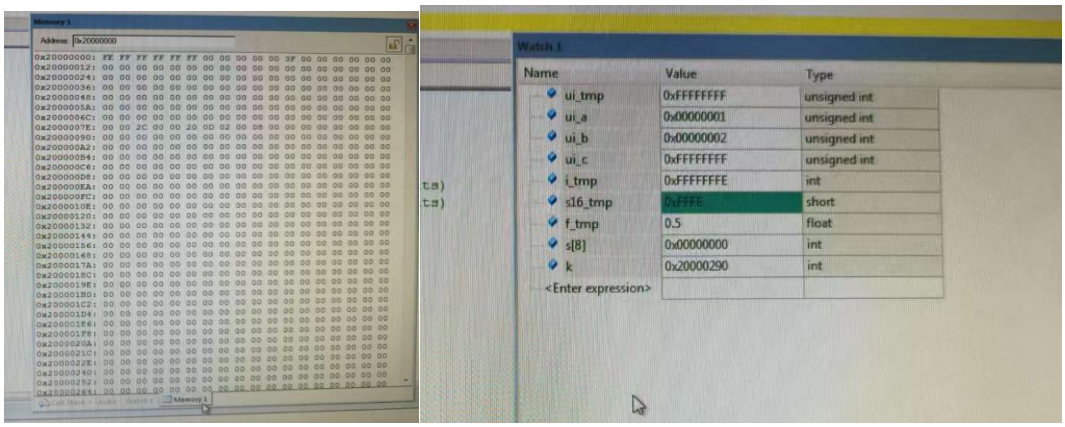

Watch 1
Name
Value
Type
ui_tmp
0xFFFFFFFF
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0xFFFFFFFF
unsigned int
i_tmp
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>

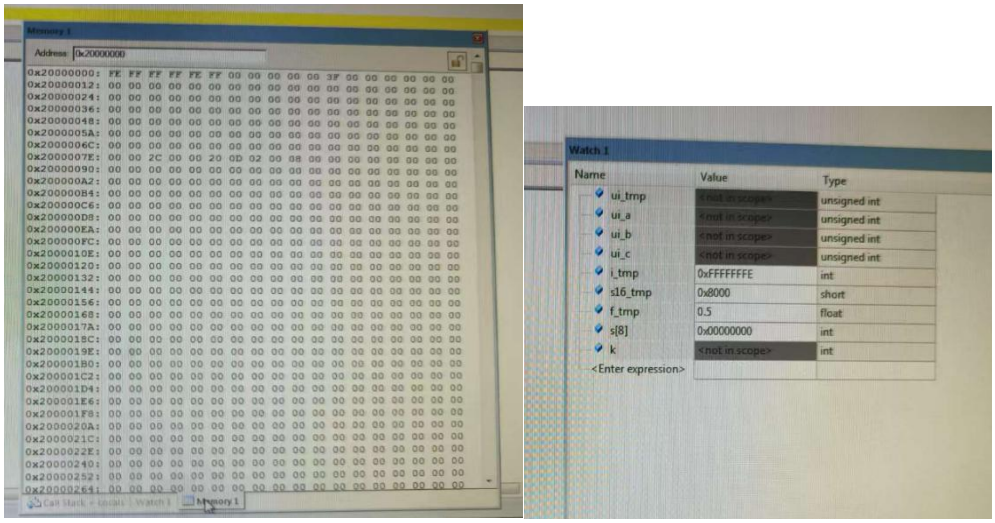

Memory 1
Address: 0x20000000
0x20000000: FF FF FF FF 00 00 00 00 00 00 00 3F 00 00 00 00 00
0x20000012: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000024: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000036: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000048: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000005A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000006C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000007E: 00 00 2C 00 00 20 0D 02 00 08 00 00 00 00 00 00 00
0x20000090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000A2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000B4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000C6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000D8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000EA: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200000FC: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000010E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000132: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000144: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000156: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000168: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000017A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000018C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000019E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001C2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001D4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001E6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x200001F8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000020A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000021C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x2000022E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000240: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000252: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x20000264: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Memory 1

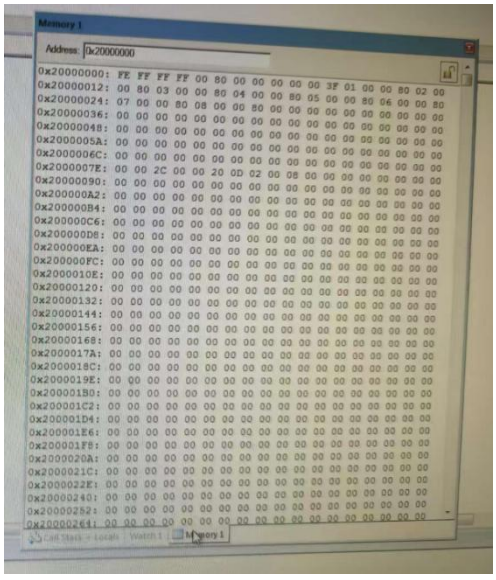

Watch 1
Name
Value
Type
ui_tmp
0xFFFFFFFF
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0xFFFFFFFF
unsigned int
i_tmp
0xFFFFFFFE
int
s16_tmp
0x0000
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>


Memory 1
Address: 0x20000000
下一张


Watch 1
Name
Value
Type
ui_tmp
0xFFFFFFFF
unsigned int
ui_a
0x00000001
unsigned int
ui_b
0x00000002
unsigned int
ui_c
0xFFFFFFFF
unsigned int
i_tmp
0xFFFFFFFE
int
s16_tmp
0xFFFF
short
f_tmp
0.5
float
s[8]
0x00000000
int
k
0x20000290
int
<Enter expression>
下一张







3. 使用 CMSIS-CORE 函数实现底层操作并记录读取内容。

阅读教材 P278~P282，特殊寄存器。单步执行程序中如下代码并记录，对比 ui_tmp 的值和 Register 窗口观察到的数值是否一致，如图 1-17 所示。

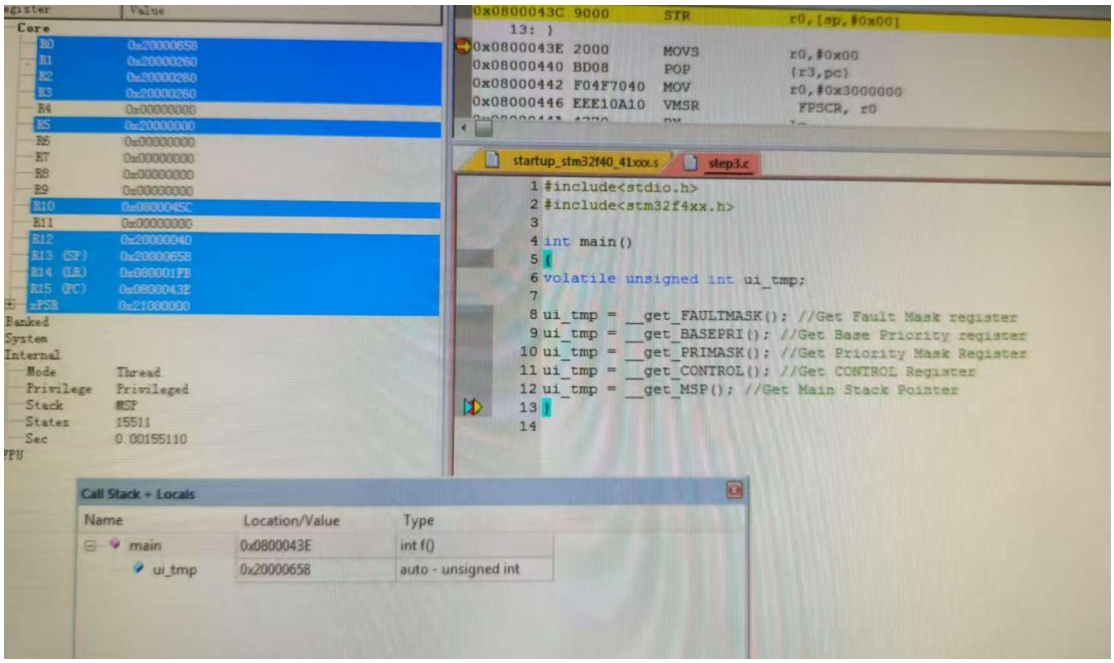
注意：相关 CMSIS-CORE 函数调用需包含头文件#include "stm32f4xx.h"

```
ui_tmp = __get_FAULTMASK(); //Get Fault Mask register
```

```
ui_tmp = __get_BASEPRI(); //Get Base Priority register
ui_tmp = __get_PRIMASK(); //Get Priority Mask Register
ui_tmp = __get_CONTROL(); //Get CONTROL Register
ui_tmp = __get_MSP(); //Get Main Stack Pointer
```

分析如下：

拍照得到下图：



由此图可知，其 ui_tmp 的值和 Register 观察的值一致。

通过在 STM32F4 平台上使用 CMSIS-CORE 函数进行底层操作，我们可以直接访问和操作一些特殊寄存器。这些寄存器与系统的中断、异常处理、优先级管理等功能密切相关。你提供的代码片段涉及了几个关键寄存器的读取，它们是：

FAULTMASK 寄存器（__get_FAULTMASK）：

用于控制是否允许非本地中断。它的值为 1 时，系统禁用所有中断（包括不可屏蔽的中断）。当读取该寄存器时，获取到的是当前是否允许中断的状态。

BASEPRI 寄存器（__get_BASEPRI）：

用于设置和读取当前处理器的基优先级。该寄存器的值表示当前允许的最低中断优先级。高于这个优先级的中断可以被处理，低于该值的中断会被屏蔽。当读取该寄存器时，返回的是当前的基优先级值。

PRIMASK 寄存器（__get_PRIMASK）：

用于控制是否允许中断。它的值为 1 时，禁用所有中断（不可屏蔽中断除外）。如果读取该寄存器，返回值为 0 表示中断启用，返回值为 1 表示禁用。

CONTROL 寄存器（__get_CONTROL）：

控制 CPU 的特权级别和堆栈指针（MSP 或 PSP）的选择。根据返回的值，可以确定 CPU 是否处于特权模式或用户模式，以及当前使用的是主堆栈指针（MSP）还是进程堆栈指针（PSP）。

MSP 寄存器（__get_MSP）：

获取当前的主堆栈指针（Main Stack Pointer）的值。堆栈指针在不同模式下（如线程模式或异常模式）可能不同。该寄存器的值即为当前堆栈的起始位置。

分析：

这些函数的作用是帮助开发者直接访问和操作系统级的寄存器，以便进行底层控制。通过这些函数，你可以控制中断的启用与禁用、优先级的设置、堆栈的选择等。读取这些寄存器并观察返回值的变化，可以帮助开发者深入理解处理器的中断管理、任务调度等机制。

关键点：

寄存器的状态影响系统行为：通过操作这些寄存器，可以直接控制系统的中断响应、处理模式以及堆栈的使用等。

工具的使用（CMSIS-CORE）：这些 CMSIS-CORE 函数的使用简化了底层操作，使得开发者能够直接访问这些特殊寄存器，避免了复杂的底层硬件操作。